part-a

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-data/master/diamonds.csv")

print(df.shape)   # should be > 50,000 rows
df.head()
```

```
(53940, 10)
```

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

Next steps: Generate code with df New interactive sheet

```
print(df.loc[0:4, ['carat', 'price']])
print(df.loc[10:15, ['cut', 'color']])
print(df.loc[:, ['price']])
```

```
   carat  price
0   0.23    326
1   0.21    326
2   0.23    327
3   0.29    334
4   0.31    335
        cut color
10     Good     J
11    Ideal     J
12  Premium     F
13    Ideal     J
14  Premium     E
15  Premium     E
       price
0        326
1        326
2        327
3        334
4        335
...      ...
53935   2757
53936   2757
53937   2757
53938   2757
53939   2757

[53940 rows x 1 columns]
```

```
print(df.iloc[0:5, 0:2])
print(df.iloc[10:20, 2:5])
print(df.iloc[0])
```

```
   carat      cut
0   0.23    Ideal
1   0.21  Premium
2   0.23     Good
```

```
3   0.29  Premium
4   0.31     Good
   color clarity  depth
10     J     SI1   64.0
11     J     VS1   62.8
12     F     SI1   60.4
13     J     SI2   62.2
14     E     SI2   60.2
15     E      I1   60.9
16     I     SI2   62.0
17     J     SI1   63.4
18     J     SI1   63.8
19     J     SI1   62.7
carat       0.23
cut        Ideal
color          E
clarity      SI2
depth       61.5
table       55.0
price        326
x           3.95
y           3.98
z           2.43
Name: 0, dtype: object
```

```
df[(df['price'] > 5000) & (df['carat'] > 1)]
df[df['cut'] == 'Premium']
df[df['price'] > 10000]
```

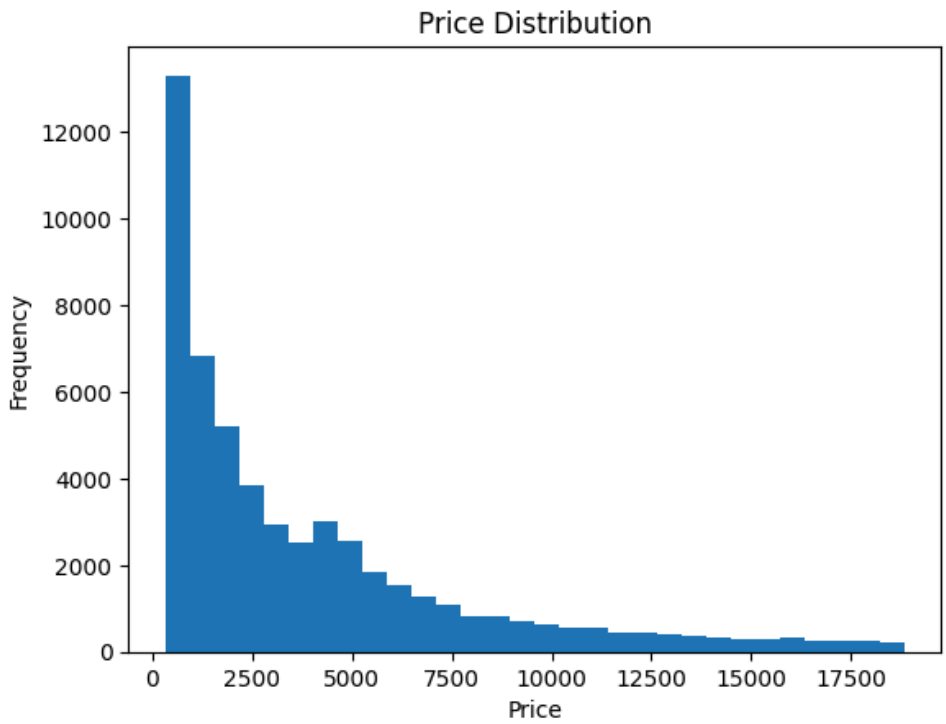
	carat	cut	color	clarity	depth	table	price	x	y	z
21928	1.70	Ideal	J	VS2	60.5	58.0	10002	7.73	7.74	4.68
21929	1.03	Ideal	E	VVS2	60.6	59.0	10003	6.50	6.53	3.95
21930	1.23	Very Good	G	VVS2	60.6	55.0	10004	6.93	7.02	4.23
21931	1.25	Ideal	F	VS2	61.6	55.0	10006	6.93	6.96	4.28
21932	2.01	Very Good	I	SI2	61.4	63.0	10009	8.19	7.96	4.96
...	...	...	...	...	...	...	...	...	...	...
27745	2.00	Very Good	H	SI1	62.8	57.0	18803	7.95	8.00	5.01
27746	2.07	Ideal	G	SI2	62.5	55.0	18804	8.20	8.13	5.11
27747	1.51	Ideal	G	IF	61.7	55.0	18806	7.37	7.41	4.56
27748	2.00	Very Good	G	SI1	63.5	56.0	18818	7.90	7.97	5.04
27749	2.29	Premium	I	VS2	60.8	60.0	18823	8.50	8.47	5.16

5222 rows × 10 columns

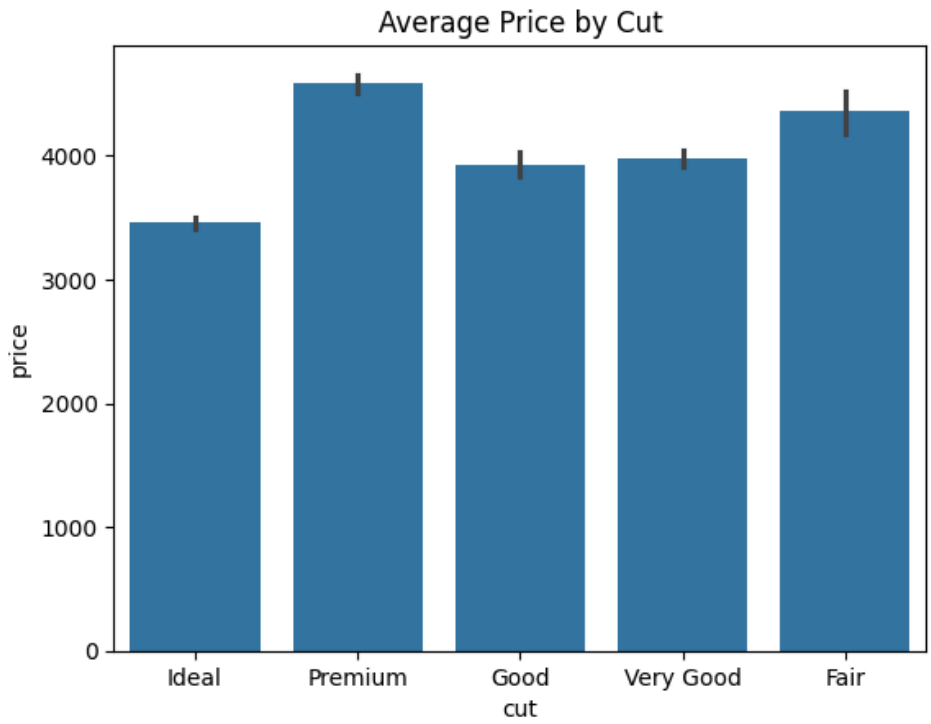
```
df.describe()
```

	carat	depth	table	price	x	y	
count	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	53940.000(
mean	0.797940	61.749405	57.457184	3932.799722	5.731157	5.734526	3.5387
std	0.474011	1.432621	2.234491	3989.439738	1.121761	1.142135	0.7056
min	0.200000	43.000000	43.000000	326.000000	0.000000	0.000000	0.000(
25%	0.400000	61.000000	56.000000	950.000000	4.710000	4.720000	2.910(
50%	0.700000	61.800000	57.000000	2401.000000	5.700000	5.710000	3.530(
75%	1.040000	62.500000	59.000000	5324.250000	6.540000	6.540000	4.040(
max	5.010000	79.000000	95.000000	18823.000000	10.740000	58.900000	31.800(

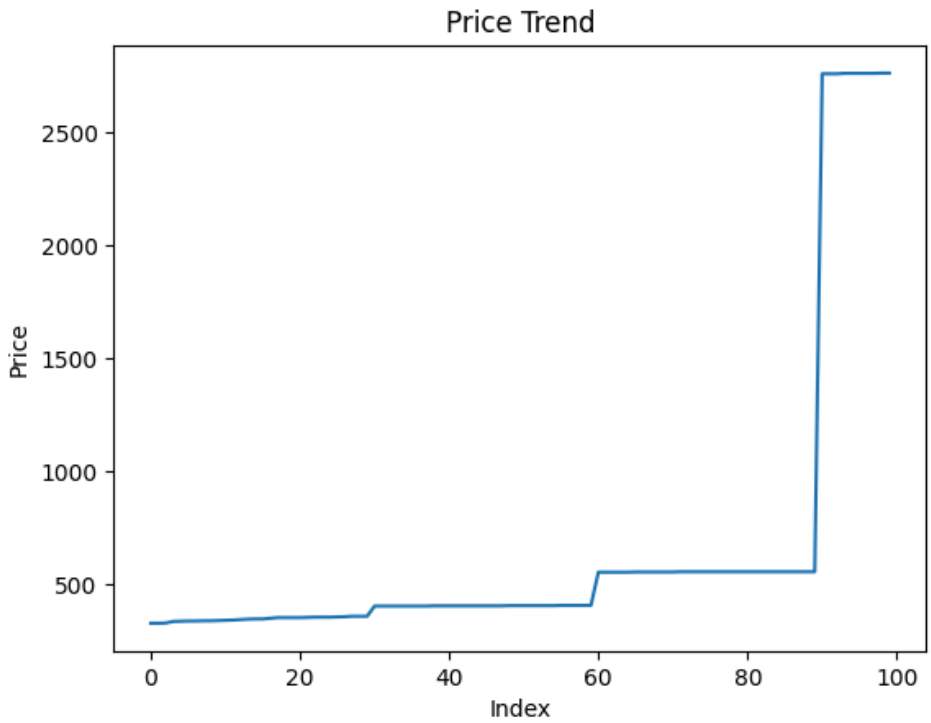
```
plt.hist(df['price'], bins=30)
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.title("Price Distribution")
plt.show()
```



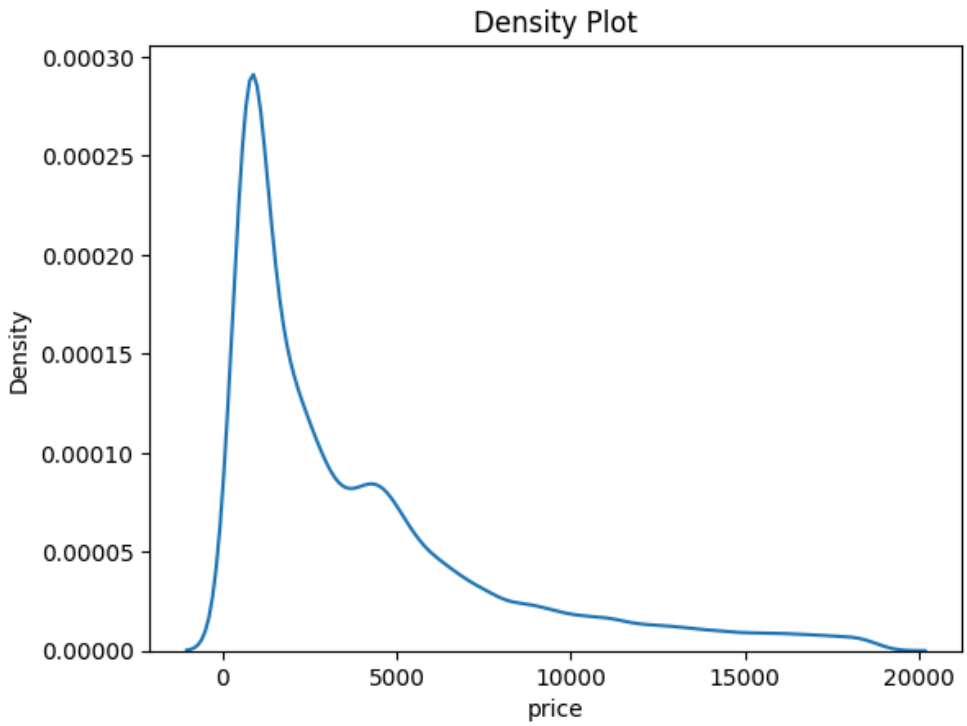

```
sns.barplot(x='cut', y='price', data=df)
plt.title("Average Price by Cut")
plt.show()
```




```
df['price'].head(100).plot()
plt.xlabel("Index")
plt.ylabel("Price")
plt.title("Price Trend")
plt.show()
```




```
sns.kdeplot(df['price'])
plt.title("Density Plot")
plt.show()
```



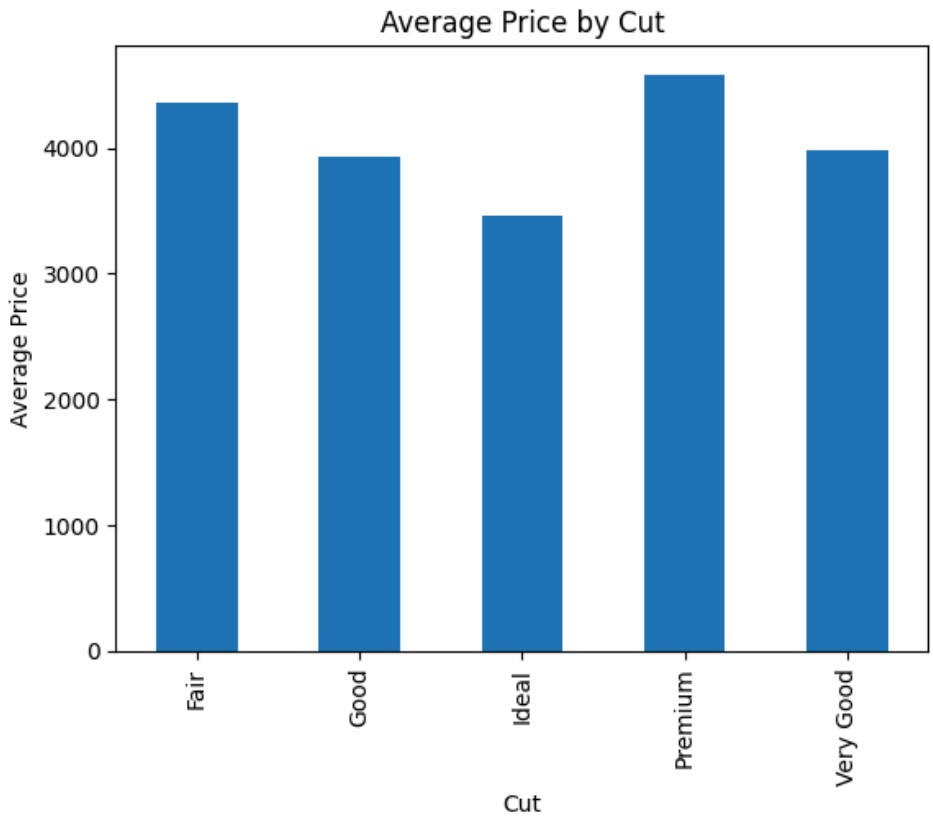
part-b

```
grouped = df.groupby('cut')['price'].mean()
grouped
```

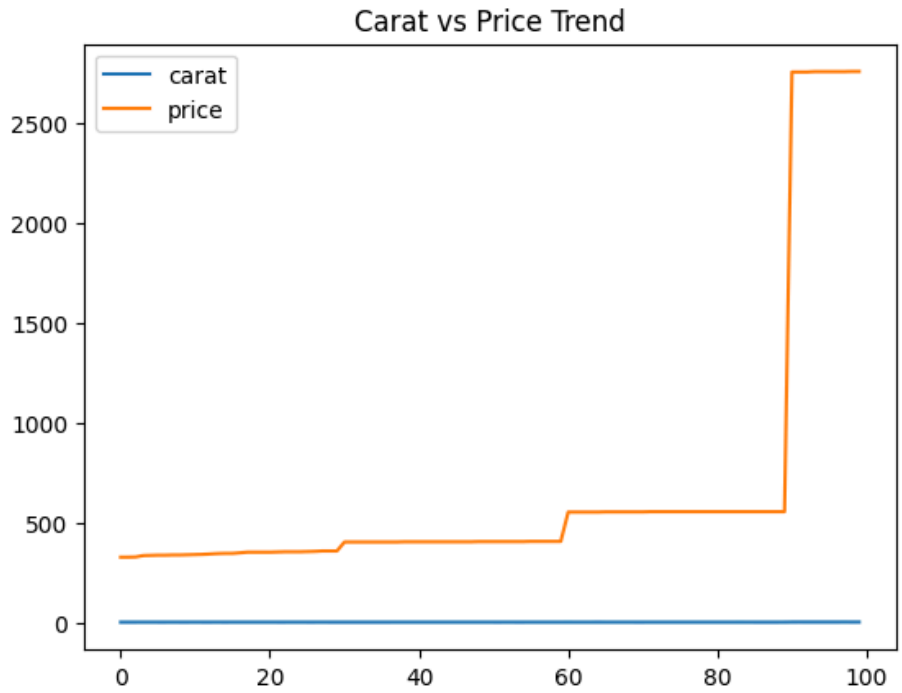
cut	price
Fair	4358.757764
Good	3928.864452
Ideal	3457.541970
Premium	4584.257704
Very Good	3981.759891

dtype: float64

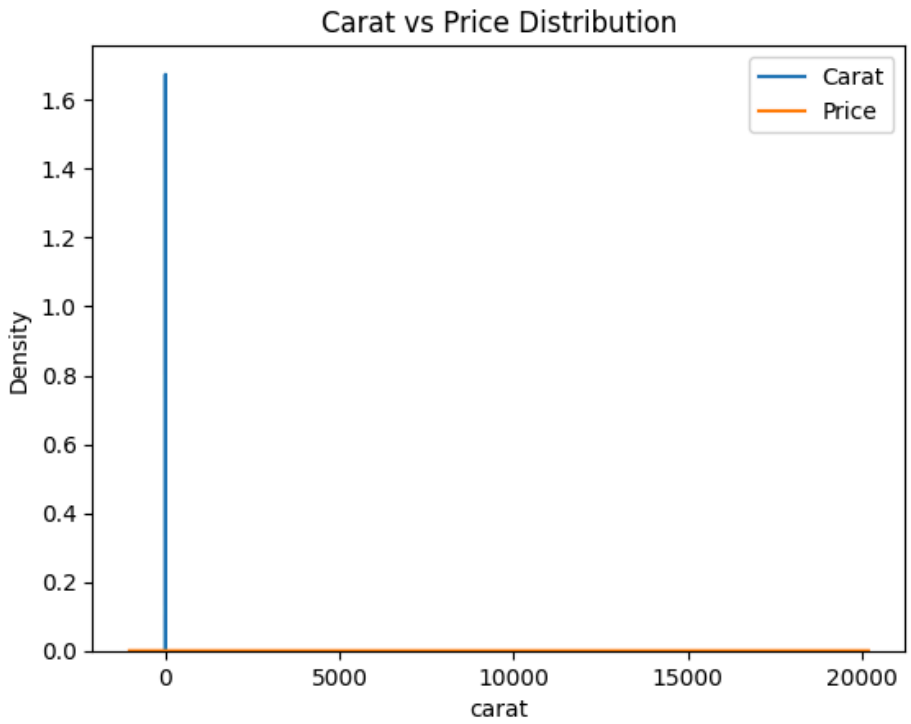
```
grouped.plot(kind='bar')
plt.xlabel("Cut")
plt.ylabel("Average Price")
plt.title("Average Price by Cut")
plt.show()
```




```
df[['carat', 'price']].head(100).plot()
plt.title("Carat vs Price Trend")
plt.show()
```




```
sns.kdeplot(df['carat'], label='Carat')
sns.kdeplot(df['price'], label='Price')
plt.legend()
plt.title("Carat vs Price Distribution")
plt.show()
```



part-c

```
avg_price = df['price'].mean()
df[df['price'] > avg_price]
```

	carat	cut	color	clarity	depth	table	price	x	y	z
5903	0.90	Premium	H	VS2	62.6	58.0	3933	6.14	6.19	3.86
5904	0.93	Very Good	G	SI1	62.5	57.8	3933	6.15	6.28	3.87
5905	0.83	Ideal	D	SI1	61.5	53.0	3933	6.09	6.07	3.74
5906	0.90	Good	E	SI1	60.8	65.0	3933	6.14	6.17	3.74
5907	0.81	Ideal	F	VS1	61.5	55.0	3933	6.01	5.96	3.68
...	...	...	...	...	...	...	...	...	...	...
27745	2.00	Very Good	H	SI1	62.8	57.0	18803	7.95	8.00	5.01
27746	2.07	Ideal	G	SI2	62.5	55.0	18804	8.20	8.13	5.11
27747	1.51	Ideal	G	IF	61.7	55.0	18806	7.37	7.41	4.56
27748	2.00	Very Good	G	SI1	63.5	56.0	18818	7.90	7.97	5.04
27749	2.29	Premium	I	VS2	60.8	60.0	18823	8.50	8.47	5.16

19657 rows × 10 columns

```
import pandas as pd

df = pd.read_csv("data.csv")

filtered = df[df['price'] > 5000]
```

```
df.head()
```

	id	price	carat	cut	color
0	1	3732	2.27	Very Good	F
1	2	11799	2.11	Fair	G
2	3	10845	1.94	Very Good	I
3	4	14123	2.19	Premium	D
4	5	10225	0.77	Ideal	G

Next steps: Generate code with df New interactive sheet

```
df.describe()
```

	id	price	carat
count	600.000000	600.000000	600.000000
mean	300.500000	10454.413333	1.584017
std	173.349358	5410.287252	0.823265
min	1.000000	1063.000000	0.200000
25%	150.750000	5916.500000	0.880000
50%	300.500000	10376.000000	1.550000
75%	450.250000	15168.500000	2.275000
max	600.000000	19994.000000	3.000000

```
filtered = df[df['price'] > 5000]
filtered
```

	id	price	carat	cut	color
1	2	11799	2.11	Fair	G
2	3	10845	1.94	Very Good	I
3	4	14123	2.19	Premium	D
4	5	10225	0.77	Ideal	G
5	6	15116	1.16	Premium	H
...	...	...	...	...	...
594	595	6851	1.65	Premium	D
595	596	15179	0.39	Very Good	F
596	597	11681	1.52	Very Good	J
598	599	15578	0.77	Good	F
599	600	5966	1.39	Premium	D

475 rows × 5 columns

Next steps: Generate code with filtered New interactive sheet

```
import matplotlib.pyplot as plt

plt.hist(df['price'])
plt.title("Price Distribution")
plt.show()
```

Price Distribution
70
60
50
40
30
20